Savitribai Phule Pune University

Department of Technology

Natural Language Processing

for Data Science

Course – BSC Data Science

Savitribai Phule Pune University

Department of Technology

BSC Data Science

SEMESTER – VI

Subject Name:

Student Name: Vaishnavi Vijay Punde

Roll No: BSC23DS121

Submission Date:

Teacher Name: Prof.Divesh Jadhwani

Internal Examiner signature	External Examiner signature	HOD signature

INDEX

SR.NO	DATE	TITLE	REMARK	SIGNATURE
1		Practical on Text Preprocessing (Tokenization, Stopword Removal)		
2		Practical on Stemming & Lemmatization		
3		Practical on Part-of-Speech (POS) Tagging		
4		Practical on Named Entity Recognition (NER)		
5		Practical on N-grams & Language Modeling		
6		Practical on TF-IDF Vectorization		
7		Practical on Word Embeddings		
8		Practical on Sentiment Analysis		

PRACTICAL NO. 1

Aim: - To perform Part-of-Speech tagging.

Objective: To identify grammatical roles of words.

Program:

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

texts = [
    "Apple is looking at buying a startup in India."
]

stop_words = set(stopwords.words('english'))

for text in texts:
    tokens = word_tokenize(text)
    filtered = [word for word in tokens if word.lower() not in stop_words]

    print("Original:", text)
    print("Tokens:", tokens)
    print("After Stopword Removal:", filtered)
```

Python Library Used : nltk

Output: -

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
Original: Apple is looking at buying a startup in India.
Tokens: ['Apple', 'is', 'looking', 'at', 'buying', 'a', 'startup', 'in', 'India', '.']
After Stopword Removal: ['Apple', 'looking', 'buying', 'startup', 'India', '.']
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

Conclusion :- Text preprocessing helps in removing unnecessary

words and splitting text into meaningful tokens. It improves the quality of input for NLP models.

PRACTICAL NO. 2

Aim: - To perform stemming and lemmatization.

Objective: To reduce words to their root forms.

Program:

```
import nltk
nltk.download('wordnet')

from nltk.stem import PorterStemmer, WordNetLemmatizer

words = ["running", "better", "flies", "writing"]

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print("Stemming:")
for w in words:
    print(w, "->", stemmer.stem(w))

print("\nLemmatization:")
for w in words:
    print(w, "->", lemmatizer.lemmatize(w))
```

Python Library Used : nltk

Output: -

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
Stemming:
running -> run
better -> better
flies -> fli
writing -> write

Lemmatization:
running -> running
better -> better
flies -> fly
writing -> writing
```

Conclusion :- Stemming is faster but less accurate, while

lemmatization gives meaningful root words using vocabulary knowledge.

PRACTICAL NO. 3

Aim: - To perform Part-of-Speech tagging.

Objective: To identify grammatical roles of words.

Program:

```
import nltk
#nltk.download('averaged_perceptron_tagger_eng')
nltk.download('averaged_perceptron_tagger')

from nltk.tokenize import word_tokenize

text = "She enjoys reading books."

tokens = word_tokenize(text)
pos_tags = nltk.pos_tag(tokens)

print("Tokens:", tokens)
print("POS Tags:", pos_tags)
```

Python Library Used : nltk

Output: -

```
Tokens: ['She', 'enjoys', 'reading', 'books', '.']
POS Tags: [('She', 'PRP'), ('enjoys', 'VBZ'), ('reading', 'VBG'), ('books', 'NNS'), ('.', '.')]
```

Conclusion :- POS tagging helps in understanding sentence structure and improves tasks like parsing and translation.

PRACTICAL NO. 4

Aim: - To identify named entities in text.

Objective: To extract names, organizations, and locations.

Program:

```
# nltk.download('maxent_ne_chunker_tab')

import nltk
nltk.download('maxent_ne_chunker')
nltk.download('words')

from nltk import word_tokenize, pos_tag, ne_chunk

text = "Apple is looking at buying a startup in India."

tokens = word_tokenize(text)
tags = pos_tag(tokens)
entities = ne_chunk(tags)

print(entities)
```

Python Library Used : nltk

Output: -

```
(S
  (GPE Apple/NNP)
  is/VBZ
  looking/VBG
  at/IN
  buying/VBG
  a/DT
  startup/NN
  in/IN
  (GPE India/NNP)
  ./.)
```

Conclusion :- POS tagging helps in understanding sentence structure and improves tasks like parsing and translation.